ISA 模拟系统 初步软件需求

> 项目组：张帛昊、张豫君、上官源博、韩中贤、黄文康
> 文档阶段：初步软件需求（需求获取阶段产出）
> 版本：v1.2（九项评审通过；修正 5.3 表格断表问题）

1 软件背景介绍

指令集体系结构（Instruction Set Architecture，ISA）是软件与硬件之间的接口契约，它规定了处理器可见的指令编码、寄存器组织、内存寻址方式与特权行为。学习者、芯片设计者、指令集制定者与底层程序开发者都需要围绕这一契约开展工作，而他们共同依赖一个前提：**能够准确、便捷地观察到指令执行后体系结构状态发生了什么变化**。

近年来以 RISC-V 为代表的开放指令集迅速普及，围绕 ISA 的教学、验证与工具需求显著上升。目前满足这一需求的手段主要是三类：阅读规范手册与教材；搭建 RTL 级仿真环境（Verilator、VCS 等）；在真实硬件或 FPGA 上运行程序。这三类手段各有其成本——手册是静态的、无法执行；RTL 仿真启动与编译开销大、反馈周期长；真实硬件则难以逐指令观察与反复回退。

本项目拟开发一个**纯软件的 ISA 模拟系统**，以解释执行的方式重建指令级行为，并提供可视化、可调试、可批量验证、可与外部参考模拟器比对的能力。系统在执行内核与指令集定义之间保留抽象边界，首期完整实现 RISC-V RV32I 指令集，支持第二套指令集作为后续演进目标。系统以 Web 形式提供，支持云端访问与本地部署两种形态，并对外提供集成 API。

项目组由 5 名成员组成，开发工具为 Visual Studio Code，核心采用 C++ 实现，版本控制使用 Git 并托管于 Gitee 或 Github。

1.1 术语与缩略语

本文档全文按下表统一用词，正文中不再出现同义异形写法。

术语 / 缩略语	含义	本文使用约定
ISA	Instruction Set Architecture，指令集体系结构	全文统一使用 "ISA"
GPR	General Purpose Register，通用寄存器	表格与图中统一使用 "GPR"；正文首次出现写 "通用寄存器（GPR）"
CSR	Control and Status Register，控制与状态寄存器	全文统一使用 "CSR"
ISA 描述文件	描述指令编码格式、指令语义与架构状态的机器可读文件	统一使用该名称，不使用 "ISA 插件""指令集描述" 等写法
外部参考模拟器	作为对拍基准的第三方成熟模拟器	统一使用该名称，不使用 "参考实现""黄金模型" 等写法

术语 / 缩略语	含义	本文使用约定
对拍	两套彼此独立的实现执行同一程序并逐条比对架构状态	全文统一使用 "对拍"
架构状态	PC、GPR、内存、CSR 等体系结构可见状态	统一使用该名称
执行轨迹	逐步记录（指令地址、指令、操作数、状态增量）	统一使用该名称
终止序列	约定的程序退出指令序列（RV32I 的 ecall / ebreak 退出约定）	统一使用该名称
比对子集	对拍时参与比对的状态集合	统一使用该名称
工业芯片工程师	选题文档所述的用户类型之一	全文统一使用 "工业芯片工程师"，不简写为 "芯片工程师"

2 欲解决的问题

2.1 指令执行过程不可观察

ISA 规范以文字和公式描述指令语义，学习者只能在脑中"执行"这些语义。现有工具中，反汇编器只做静态翻译，调试器面向的是真实机器上的完整程序，都无法把"某一条指令执行前后，PC、GPR、内存、CSR 分别发生了什么"以即时、可视的方式呈现。

影响：学习曲线陡峭，概念性错误难以自查。

2.2 指令级功能验证成本高

芯片前端团队需要在 RTL 开发前或开发中对指令功能做冒烟级验证。当前做法依赖 EDA 工具链拉起完整仿真环境，编译与初始化开销远高于"跑几条指令看看结果"这一实际需求。

影响：验证迭代慢，简单问题被高成本流程掩盖；小规模验证需求无法被满足。

2.3 指令集规范本身的歧义难以发现

若只有一套实现，实现与规范之间的偏差无法被察觉——实现错了会被当成规范，规范本身自相矛盾之处也无人暴露。发现规范歧义的经典手段是让两套彼此独立的实现执行同一程序并比对结果，而业界缺少面向教学的、开箱即用的对拍工具。

影响：规范缺陷被推迟到流片或编译器适配阶段才暴露，修改代价高昂。

2.4 机器语言与汇编程序缺乏轻量调试环境

直接编写机器码或汇编的程序员，需要一个可以设置断点、单步执行、查看内存与寄存器，并能回退重放的环境。面向通用应用的高级语言调试器无法覆盖裸机场景。

影响：底层程序的调试效率低，错误定位依赖打印与猜测。

2.5 问题与相关方对应关系

问题	受影响相关方
2.1 执行过程不可观察	ISA 学习者
2.2 指令级功能验证成本高	工业芯片工程师
2.3 规范歧义难以发现	ISA 工程师
2.4 裸机调试环境缺失	机器语言程序员

---

3 软件解决方案

3.1 总体思路

以纯软件的**解释执行内核**重现指令级行为，把"架构状态"变成可以被程序读取、展示、回放和比对的数据对象；在内核之上叠加**服务层（集成 API）**与**Web 前端（可视化与交互）**，形成"内核可复用、界面可替换、能力可对外集成"的分层结构。指令的格式与语义以 **ISA 描述文件**形式提供，内核与指令集定义之间保持抽象边界；首期以 RISC-V RV32I 完整实现并验证该边界成立。

3.2 系统组成与职责划分

组成	由谁提供	承担职责
模拟执行内核	本软件开发	取指—译码—执行—写回循环；维护 PC、GPR、内存、CSR 等架构状态；记录执行轨迹；识别非法与未定义编码；处理程序终止序列与运行期异常
ISA 抽象层与描述装载	本软件开发	定义内核与 ISA 描述文件之间的抽象边界；装载并校验 ISA 描述文件；首期提供 RISC-V RV32I 描述
服务端与集成 API	本软件开发	用户与会话管理；调试控制（单步、运行、暂停、断点）；测试用例编排与判定；对拍调度与差异汇总；对外提供 HTTP/WebSocket 接口
Web 前端	本软件开发	寄存器、内存、反汇编、执行轨迹四类视图；单步与断点交互；测试报告与对拍差异报告呈现；示例程序与新手引导入口
示例程序集	本软件开发	提供覆盖 RV32I 常用指令的演示程序与经典算法样例，支撑快速上手
外部汇编工具链	第三方提供（复用）	将汇编源转换为机器码；工具链缺失时由用户自行提供机器码
外部参考模拟器	第三方提供（复用）	作为对拍基准执行同一程序并输出逐步架构状态。首期锁定 **Spike**，利用其 `--log-commits` 逐步提交日志；QEMU 等列为后续扩展目标
浏览器与宿主操作系统	环境提供	提供渲染容器与运行环境（云主机或本地主机）
Git 托管平台	环境提供	源代码、测试用例集与文档的版本管理与协作

3.3 系统间的交互

- **前端 ↔ 服务端**：前端经 HTTP/WebSocket 调用服务端接口，发起装载、单步、运行、测试、对拍等请求，接收架构状态增量推送与结果数据。

- **服务端 ↔ 外部汇编工具链**：服务端以子进程方式调用工具链完成汇编，读取其产物与错误输出；工具链不可用时返回明确提示，不静默失败。
- **服务端 ↔ 外部参考模拟器**：服务端以子进程方式启动 Spike 执行程序，采集其逐步提交日志作为对照状态，供对拍使用。
- **服务端 ↔ 存储**：用户工作区（程序、用例集、配置）持久化到用户私有空间，不同用户之间互相隔离。
- **本地部署形态**：服务端与前端以容器或可执行文件形式部署在本地主机，功能与云端一致。

3.4 业务目标

编号	业务目标	对应问题	来源
G1	让 ISA 学习者在无硬件条件下，以可视化单步的方式理解指令语义	2.1	S1、S3
G2	让工业芯片工程师在秒级完成指令级功能冒烟验证，不依赖 EDA 工具链	2.2	S1、S4
G3	让 ISA 工程师通过两套独立实现的行为对拍，发现规范本身的歧义与不一致	2.3	S1、S5
G4	让机器语言程序员在可回退、可重放的模拟环境中调试裸机程序	2.4	S1、S6

3.5 范围与边界

本期功能范围（本系统负责）：ISA 描述装载与校验（首期 RISC-V RV32I）、程序装载与运行配置、指令解释执行、程序终止与运行期异常处理、指令级单步与断点调试、寄存器与内存可视化、示例程序与新手引导、测试用例组织与批量验证、与 Spike 的一致性对拍、集成 API、多用户会话与隔离。

本期不在范围内：

排除项	说明
周期精确的微架构建模	不建模流水线、缓存时序等微架构行为，仅做体系结构级（功能级）模拟
完整外设与中断建模	仅处理约定的终止序列与运行期异常，不建模完整的中断控制器与外设
操作系统级全系统仿真	不运行操作系统，面向裸机程序
第二套指令集的完整实现	本期仅在架构上保留 ISA 抽象边界，不要求第二套 ISA 真正跑通
QEMU 等其他外部参考模拟器的接入	本期对拍基准锁定 Spike，其余参考模拟器列为后续演进目标

边界划分：

由本软件实现	由环境或第三方提供
指令语义解释、架构状态维护、调试控制、状态可视化、示例程序、测试编排与判定、对拍调度与差异归因、API 服务、用户与会话管理	浏览器渲染容器、宿主操作系统、外部汇编工具链、外部参考模拟器（Spike）、Git 托管服务

4 软件的功能需求描述

4.1 执行者

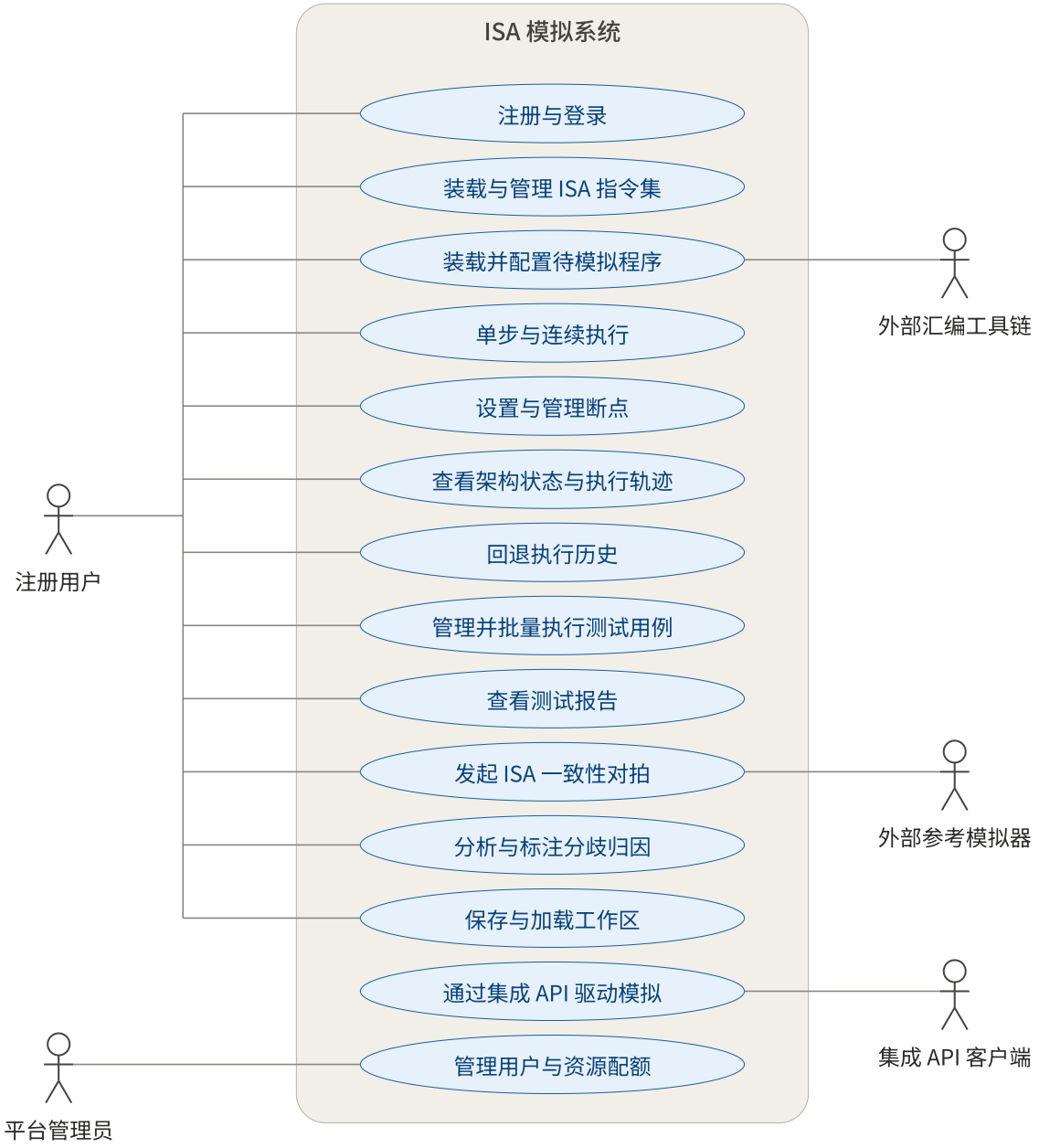
人类执行者

执行者	说明
注册用户（抽象执行者）	所有已登录用户的共性行为集合
└ ISA 学习者	泛化自注册用户，关注指令语义的可视化理解
└ 工业芯片工程师	泛化自注册用户，关注批量测试与验证报告
└ ISA 工程师	泛化自注册用户，关注 ISA 描述管理与对拍归因
└ 机器语言程序员	泛化自注册用户，关注断点调试与裸机程序运行
平台管理员	负责用户管理与资源配额管理

系统执行者（次要执行者）：外部汇编工具链、外部参考模拟器（Spike）、集成 API 客户端。浏览器与宿主操作系统作为环境执行者，不参与用例划分。

> **说明（需求验证状态）**：上表四类具体用户画像来自项目选题文档（来源 S1），**尚未经真实用户访谈确认**，其中"工业芯片工程师"的作业方式（是否接受 Web 形态的工具，抑或更倾向脚本与 CI）建议后续核实。核实结论可能影响 4.2 的用例构成与 NFR-Q4 的形态要求。

4.2 用例图



> 图中四种具体用户类型（ISA 学习者、工业芯片工程师、ISA 工程师、机器语言程序员）泛化自"注册用户"，为保持图面清晰已合并表示；«include» / «extend» 关系见 4.4 节；关联线采用总线汇聚画法，仅表示执行者与用例的连接，不代表用例之间存在依赖。

4.3 用例列表与基本交互序列

用例	主要执行者	基本交互序列	来源
UC-01 注册与登录	注册用户	① 提交注册或登录凭据 → ② 系统校验并建立会话与隔离工作区 → ③ 载入默认 ISA 描述 → ④ 进入主界面	S2-6
UC-02 装载与管理 ISA 指令集	ISA 工程师	① 上传 ISA 描述文件 → ② 系统校验其完备性与自洽性 → ③ 校验通过则装载并切换为当前 ISA，不通过则返回问题清单	S2-1、S5

用例	主要执行者	基本交互序列	来源
UC-03 装载并配置待模拟程序	注册用户	① 提交汇编源或机器码（亦可选用内置示例程序）→ ② 汇编源经外部工具链转码 → ③ 按指定基址载入内存 → ④ 设定起始 PC、栈指针与内存初值 → ⑤ 进入就绪态	S2-5、S3、S6
UC-04 单步与连续执行	注册用户	① 触发单步执行或连续运行 → ② 内核取指、译码、执行、写回 → ③ 更新 PC、GPR、内存与 CSR → ④ 记录执行轨迹并推送状态增量 → ⑤ 前端高亮本步变化 → ⑥ 命中终止序列或运行期异常时按约定结束运行并返回结束状态	S1
UC-05 设置与管理断点	注册用户	① 在反汇编视图或地址栏指定地址设置断点 → ② 系统登记断点 → ③ 运行中命中断点则暂停并高亮 → ④ 用户可启用、禁用或删除断点	S3
UC-06 查看架构状态与执行轨迹	注册用户	① 系统按步推送架构状态 → ② 渲染寄存器、内存、反汇编与执行轨迹四类视图 → ③ 用户可跳转地址、搜索内存、点击轨迹定位到某一步	S1、S3
UC-07 回退执行历史	注册用户	① 用户选择历史中的某一步 → ② 系统从执行轨迹中恢复该步的完整架构状态 → ③ 全部视图同步刷新	S3
UC-08 管理并批量执行测试用例	工业芯片工程师	① 组织或导入用例集（程序与期望态）→ ② 触发批量执行 → ③ 逐用例运行并按选定判定模式判定 → ④ 收集结果	S4
UC-09 查看测试报告	工业芯片工程师	① 系统汇总通过率与失败清单 → ② 用户点击失败用例查看失败现场状态快照与首个断言差异点	S4
UC-10 发起 ISA 一致性对拍	ISA 工程师	① 选定程序与比对子集 → ② 本系统内核与外部参考模拟器（Spike）执行同一程序 → ③ 按比对子集逐条比对架构状态 → ④ 定位并报告首个分歧点	S2-3、S1
UC-11 分析与标注分歧归因	ISA 工程师	① 查看分歧现场与差异字段 → ② 判定归类为实现缺陷、基准偏差或规范歧义 → ③ 标注结论并归档	S5
UC-12 保存与加载工作区	注册用户	① 保存程序、用例集与配置 → ② 持久化到用户私有空间 → ③ 下次登录时加载恢复	S3、S6
UC-13 通过集成 API 驱动模拟	集成 API 客户端	① 携带凭据调用接口 → ② 系统鉴权并检查配额 → ③ 执行装载、单步、测试或对拍 → ④ 返回结构化结果	S2-2
UC-14 管理用户与资源配额	平台管理员	① 查看用户列表与资源占用 → ② 调整配额或停用账号 → ③ 系统生效	S2-6、S7

4.4 用例间关系

关系	说明	来源
UC-08 «include» UC-04	批量测试必须复用同一执行内核	由需求结构导出
UC-10 «include» UC-04	一致性对拍必须复用同一执行内核	由需求结构导出
UC-09 «extend» UC-08	扩展条件：用例集中存在失败用例	由需求结构导出
UC-11 «extend» UC-10	扩展条件：对拍过程中出现分歧	由需求结构导出
UC-03 «include» 调用外部工具链	仅在提交"汇编源"的分支触发，提交机器码分支不触发	S2-5

4.5 功能需求条目

编号	功能需求	来源
FR-1.1	内核与 ISA 定义之间保持抽象边界：指令编码格式、指令语义与架构状态以 ISA 描述文件形式提供，装载后由内核统一解释执行。本期以 RISC-V RV32I 验证该边界成立，不要求同时支持第二套 ISA	S2-1、S2-8
FR-1.2	装载 ISA 描述文件时校验其完备性与自洽性（语义缺失、编码冲突、字段重叠），输出问题清单	S5
FR-1.3	支持查询、列出与切换已装载的 ISA；首期提供 RISC-V RV32I 描述	S2-1、S2-8
FR-2.1	载入机器码文件（bin、hex，以及 ELF 中标记为可装载的段）到指定基址	S6
FR-2.2	载入汇编源文件，经外部工具链汇编后载入；汇编失败时报告出错行；亦接受用户直接提供的机器码；工具链缺失时给出明确提示而不静默失败	S2-5
FR-2.3	设定起始 PC、栈指针与内存初值	S6
FR-2.4	提供内置示例程序集，覆盖 RV32I 常用指令的演示程序与经典算法样例（如寄存器运算演示、冒泡排序、斐波那契数列），并配套新手引导	S3
FR-3.1	执行取指—译码—执行—写回循环，维护 PC、GPR、内存与 CSR	S1
FR-3.2	识别非法、未定义与保留编码，定位到具体指令地址且不导致系统崩溃	S5
FR-3.3	记录执行轨迹（指令地址、指令、操作数、状态增量），供回放与比对使用	S1

编号	功能需求	来源
FR-3.4	实现程序终止与运行结束判定：识别终止序列（RV32I 的 ecall / ebreak 退出约定），设置终止标志，向调试流程与测试流程返回结束状态与退出码	S1、S4
FR-3.5	处理运行期异常：内存越界访问、非法访存地址、非法 CSR 访问等，须记录异常现场（异常类型、触发地址、当时架构状态）并按配置终止执行或暂停，不得使服务进程崩溃	S5、S9
FR-4.1	支持单步执行、连续运行、暂停与复位	S1
FR-4.2	支持按地址设置、删除、启用与禁用断点，命中后暂停并高亮	S3
FR-4.3	支持"运行到指定地址"与"运行至程序结束或触碰边界"	S6
FR-4.4	支持执行历史回退，可回到任意已执行步的架构状态	S3
FR-5.1	提供寄存器视图（PC、GPR、CSR），高亮本步发生变化的位	S1
FR-5.2	提供内存视图，按字节 / 半字 / 字显示，支持地址跳转、搜索与编辑	S1
FR-5.3	提供反汇编视图，显示当前 PC 前后若干条指令并高亮当前指令	S3
FR-5.4	提供执行轨迹与时间轴视图，点击可定位到对应步	S3
FR-6.1	支持导入与编辑测试用例（程序加期望态），支持分组与标签	S4
FR-6.2	支持批量执行用例集并自动判定通过或失败	S4
FR-6.3	输出测试报告：通过率、失败清单与失败现场状态快照	S4
FR-6.4	支持外部脚本或 CI 经 API 触发批量测试并取回结果	S2-2
FR-6.5	测试期望态同时支持"终态快照"与"逐条提交日志"两种判定模式，二者复用同一比对引擎	S2-7
FR-7.1	调用外部参考模拟器执行同一程序并获取其逐步架构状态；首期对接 Spike 的逐步提交日志	S2-3、S1
FR-7.2	按可配置的比对子集逐条比对架构状态。默认比对子集包含：① PC 值；② 32 个 GPR 的值；③ 该指令引发的内存写（写入地址与写入值）；④ 被该指令修改的 CSR 字段。用户可裁剪子集（例如仅比对 PC 与 GPR）	S2-3、S5

编号	功能需求	来源
FR-7.3	报告首个分歧点，包含指令序号、双方状态与差异字段	S5
FR-7.4	提供差异归因辅助，区分实现缺陷、基准偏差与规范歧义，支持人工标注	S5
FR-7.5	提供外部参考模拟器不可用时的降级模式，退化为基于期望态快照的断言比对	S9
FR-8.1	提供 HTTP/WebSocket 接口，支持装载程序、单步 / 运行 / 暂停、读取状态、执行测试与发起对拍	S2-2
FR-8.2	接口具备鉴权与请求限额能力，本地部署时可关闭	S2-2、S2-6
FR-8.3	提供接口文档与调用示例	S6、S8
FR-9.1	支持创建与销毁运行会话，会话之间状态隔离，异常会话可回收	S2-6
FR-9.2	支持本地单机部署与云端部署两种形态，功能对等	S2-2
FR-9.3	支持程序与用例集的保存与加载	S3、S6
FR-9.4	支持用户注册、登录、登出与会话管理	S2-6
FR-9.5	实现多用户隔离，运行环境、工作区数据、程序与用例互不可见，并按用户设置资源配额	S2-6

5 软件的非功能需求描述

5.1 外部质量需求

编号	类别	需求描述	来源
NFR-Q1	性能	单步操作端到端响应：本地部署不超过 100 毫秒，云端部署不超过 300 毫秒（不含网络传输）。连续执行吞吐：关闭轨迹记录时不低于 10^7 条 RV32I 指令每秒；开启逐步轨迹记录时不低于 10^6 条每秒	S3、S4、S9
NFR-Q2	可靠性	连续执行 10^8 条指令不出现架构状态漂移与内存泄漏；批量测试中单个用例执行失败不影响其余用例	S4
NFR-Q3	易用	新用户在 5 分钟内完成"装载程序 → 单步执行 → 观察到寄存器变化"，该目标由 FR-2.4 的内置示例程序集与新手引导支撑；架构状态的变化必须有视觉高亮	S3

编号	类别	需求描述	来源
	性	提示	
NFR-Q4	兼容性	支持 Chrome、Edge、Firefox 的最近两个大版本；最低适配 1280×720 视口	S3、S4、S5、S6
NFR-Q5	安全性	云端部署时接口需账号鉴权；被模拟程序视为不可信输入，须限制指令执行条数与内存用量，防止死循环或越界访问导致服务不可用；用户之间数据隔离；空闲会话超时回收	S2-6

5.2 内部质量需求

编号	类别	需求描述	来源
NFR-I1	可扩展性	内核与 ISA 定义之间保持抽象边界，新增 ISA 时不需修改内核的核心执行逻辑；本期以 RV32I 验证该边界成立，第二套 ISA 为后续演进目标；比对子集可配置	S2-1、S2-8
NFR-I2	可测试性	模拟内核与 Web 前端解耦，内核可独立编译为库并做单元测试	S9
NFR-I3	可移植性	内核使用 C++17 实现，可在 Linux 与 Windows 上构建，不依赖特定 IDE 或图形界面	S1、S9

5.3 软件开发约束需求

编号	类别	约束描述	来源
NFR-C1	技术栈	核心使用 C++ 实现，前端采用 Web 技术；开发工具为 Visual Studio Code	S1
NFR-C2	版本控制	使用 Git 进行版本控制，仓库托管于 Gitee 或 Github	S1
NFR-C3	团队与工期	项目组 5 人，须在课程周期内完成	S1、S8
NFR-C4	许可合规	外部汇编工具链与外部参考模拟器（首期为 Spike）及所用第三方库的开源许可证，须与本项目的分发方式兼容	S2-3、S8
NFR-C5	交付物	需求文档、设计文档、源代码与测试报告	S8

6 可行性及潜在风险

（本节预留，待项目组补充经济、技术、进度等方面的可行性分析与风险识别。）

附录 A 需求来源编号表

编号	来源	说明
S1	项目选题文档	《项目选题.docx》，由项目组提交
S2	团队决策（需求获取阶段）	见下表 S2-1 ~ S2-8
S3	需求方 · ISA 学习者	来源 S1 所述用户类型，待真实用户确认
S4	需求方 · 工业芯片工程师	来源 S1 所述用户类型，待真实用户确认
S5	需求方 · ISA 工程师	来源 S1 所述用户类型，待真实用户确认
S6	需求方 · 机器语言程序员	来源 S1 所述用户类型，待真实用户确认
S7	平台管理员	由 S2-6 派生
S8	课程要求	软件工程课程大作业的交付与工期要求
S9	技术约束	由技术选型与运行环境派生

团队决策明细

编号	决策内容
S2-1	采用多 ISA 可插拔架构作为设计方向
S2-2	交付形态为 Web 云端可访问 + 本地可部署 + 对外集成 API
S2-3	对拍基准采用外部成熟参考模拟器
S2-4	需求获取采取逐阶段确认的推进节奏
S2-5	汇编源复用外部工具链，同时允许用户直接提供机器码
S2-6	云端部署需支持多用户隔离（引入账号与会话体系）
S2-7	测试期望态支持终态快照与逐条提交日志两种判定模式
S2-8	范围收敛：首期仅完整实现 RISC-V RV32I，第二套 ISA 列为后续演进目标；对拍基准首期锁定 Spike

> **记**：S2-1 与 S2-8 存在演进关系——S2-1 确定了架构方向，S2-8 在工期约束下将"可插拔"由交付指标收敛为架构预留。引用时以 S2-8 为准。